- A `receive` which executes its `after` clause "happens before" a subsequent `send` which sends a message that it can consume.

There are also other race-prone primitives in Erlang, but it is beyond the scope of this article to describe how they interact.

Concuerror uses a *vector clock* [Mattern 1989] for each process at each state, to calculate the happens before relation for any two events. The calculation of the vector clocks uses the ideas presented in the original DPOR paper [Flanagan and Godefroid 2005b]. The only special case is for the association of a `send` with a `receive`, where we instrument the message itself with the vector clock of the sending process.

11 EXPERIMENTS

We report experimental results that compare the performance of the three DPOR algorithms that, for convenience, we will refer to as 'classic' (for the algorithm of Flanagan and Godefroid [2005a]), 'source' and 'optimal'. We ran all benchmarks on a desktop with an i7-3770 CPU (3.40 GHz), 16GB of RAM running Debian Linux 3.2.0-4-amd64. The machine has four physical cores, but presently Concuerror uses only one of them. In all benchmarks, Concuerror was started with the option `-p inf` (use "infinity" as preemption bound), which instructs the tool to explore traces without limiting the number of preemptions in each explored trace, i.e., to verify these programs.[2]

Performance on two "standard" benchmarks. First, we report performance on the two benchmarks from the DPOR paper [Flanagan and Godefroid 2005b]: filesystem and indexer. These are benchmarks that have been used in the literature to evaluate another DPOR variant (DPOR-CR of Saarikivi et al. [2012]) and a technique based on unfoldings by Kähkönen et al. [2012]. As both programs use locks, we had to emulate a locking mechanism using Erlang. To make this translation we used particular language features for these two benchmarks.

filesystem This benchmark uses two lock-handling primitives, called `acquire` and `release`. The assumptions made for these primitives are that an `acquire` and a `release` operation on the same lock are never co-enabled and should therefore not tried to be interleaved in a different way than they occur. Thus, `acquire`s are the only operations that can be swapped, if possible, to get a different interleaving.

We implemented the lock objects in Erlang as separate processes. To acquire the lock, a process sends a message with its identifier to the "lock process" and waits for a reply. Upon receiving the message, the lock process uses the identifier to reply and then waits for a `release` message. Other `acquire` messages are left in the lock's mailbox. Upon receiving the `release` message the lock process loops back to the start, retrieving the next `acquire` message and notifying the next process. This behavior can be implemented in Erlang using two selective `receive`s.

indexer This benchmark uses a CAS primitive instruction to check whether a specific entry in a matrix is zero and set it to a new value. The "Erlang way" to do this, is to try to execute an `insert_new` operation on an ETS table: if another entry with the same key exists the operation returns `false`; otherwise the operation returns `true` and the table now contains the new entry. Two `insert_new` operations on the same key are always dependent.

Both benchmarks are parametric on the number of threads they use. For filesystem we used 14, 16, 18 and 19 threads. For indexer we used 12 and 15 threads.

[2]The version of Concuerror that we used to obtain the experimental results of this section is available at: https://github.com/aronisstav/Concuerror/releases/tag/JACM_submission.